

SD202 – Circuitos Digitais IV

A-408 Arquiteturas de hardware para Convolutional Neural Networks (CNNs)

Autores	
Anderson Max Almeida e Porfírio Hbr Campinas	Nome 2 (INSTITUIÇÃO)
Nome 3 (INSTITUIÇÃO)	Nome 4 (INSTITUIÇÃO)

Histórico de revisões		
XX/XX/202X	V1.0	Versão inicial

Tópicos

- Estrutura do Material Didático

Página em branco.

Aula 16 - 17 / 18 - 19

Introdução

- A CNN (Convolutional Neural Network) é uma parte da Inteligência Artificial muito usada em reconhecimento de imagens, visão computacional e processamento de sinais.
- Ela se baseia em redes neurais artificiais e aplica filtros (kernels) e funções de ativação em diferentes camadas, com o objetivo de aprender padrões que permitam identificar e classificar imagens.
 - Camada de convolução
 - aplica filtros para detectar padrões locais (bordas, texturas).
 - Função de ativação (ex: ReLU)
 - introduz não-linearidade, mantendo apenas ativações positivas.
 - Pooling
 - reduz o tamanho da matriz, resumindo as informações importantes.
 - Camadas totalmente conectadas (Fully Connected)
 - combinam os padrões extraídos para gerar a decisão final (ex: “essa imagem é um gato”).

Introdução

- A CNN interpreta a imagem como uma matriz de números.

- Uma matriz de pixels

- Matriz $L \times A \times P$

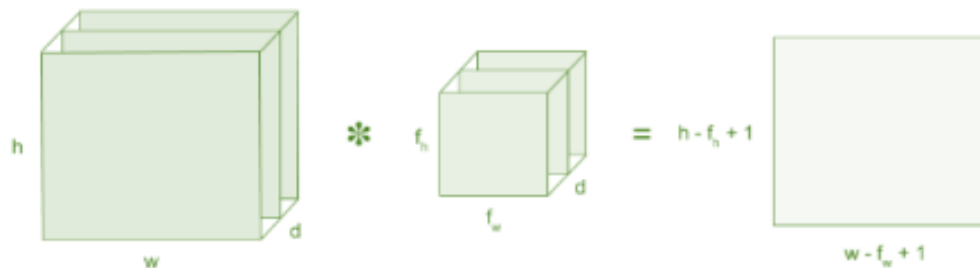
- Uma imagem de $28 \times 28 = 784$ pixels = 2354 valores.

- Em escala de cinza, a dimensão da profundidade é 1.

- An image matrix (volume) of dimension **$(h \times w \times d)$**
- A filter **$(f_h \times f_w \times d)$**
- Outputs a volume dimension **$(h - f_h + 1) \times (w - f_w + 1) \times 1$**



(RGB) - 16M de tons



Saída = $(Rpatch \times Rkernel) +$
 $(Gpatch \times Gkernel) +$
 $(Bpatch \times Bkernel)$

CNN (Convolutional Neural Network)

- Nos modelos de deep learning com CNN para treino e teste, cada imagem de entrada passa por uma série de camadas de convolução com filtros (kernels), camadas de pooling, camadas totalmente conectadas (FC) e, ao final, aplica-se a função Softmax para classificar o objeto com valores probabilísticos entre 0 e 1.
- A convolução nada mais é do que operações matemáticas entre a matriz de entrada e um filtro (Kernel) que é uma matriz menor que a analisada, varrendo pixel a pixel, até passar por toda imagem.
- O resultado é uma matriz menor ou de mesmo tamanho, com características ressaltadas pelo filtro.
- Esse destaque/ressalte de detalhes da imagem vai fazer um melhor treinamento da rede neural.

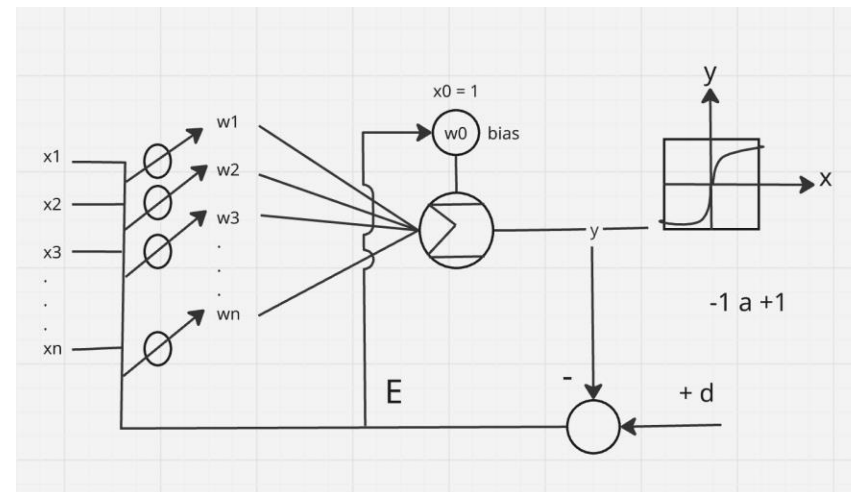
CNN (Convolutional Neural Network)

- **Esse destaque/ressalte de detalhes da imagem vai fazer um melhor treinamento da rede neural.**
- No treinamento, momento de aprendizagem da máquina, temos que enviar dados para que a rede aprenda o padrão.
- As redes artificiais modernas, MLP (Multilayer Perceptron) usam a base do Perceptron / Adaline.

$$y = \sum w_i * x_i + b$$

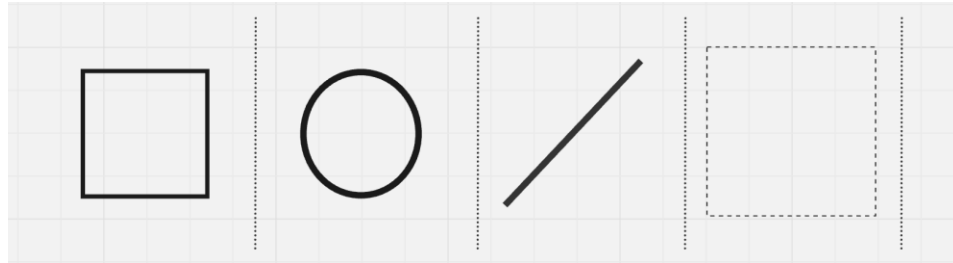
$$e = d - y$$

$$y = wx + b \text{ (Simplificado)}$$



Rede Neural Artificial

- "Rede" Neural Artificial (ANN) com 4 neurônios e 16 dados de entrada



D1 = [1,1,1,1, 1,0,0,1, 1,0,0,1, 1,1,1,1] -> y = 0001

D2 = [0,1,1,0, 1,0,0,1, 1,0,0,1, 0,1,1,0] -> y = 0010

D3 = [0,0,0,1, 0,0,1,0, 0,1,0,0, 1,0,0,0] -> y = 0100

D4 = [0,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,0] -> y = 1000

$$Y1 = W_1^t * X + b_1 = [w_{11}, w_{12}, w_{13}, \dots, w_{116}]^t * X + b_1$$

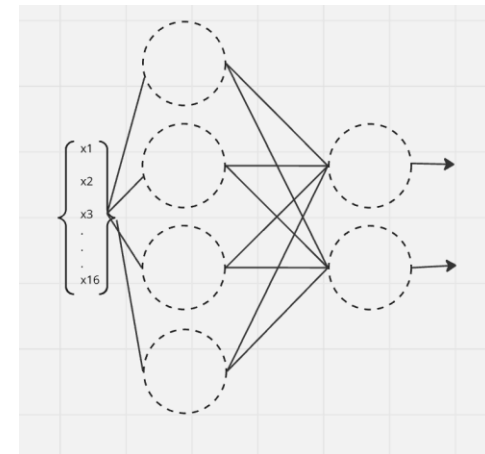
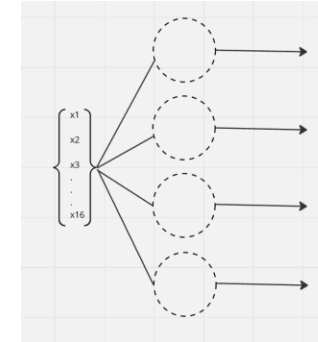
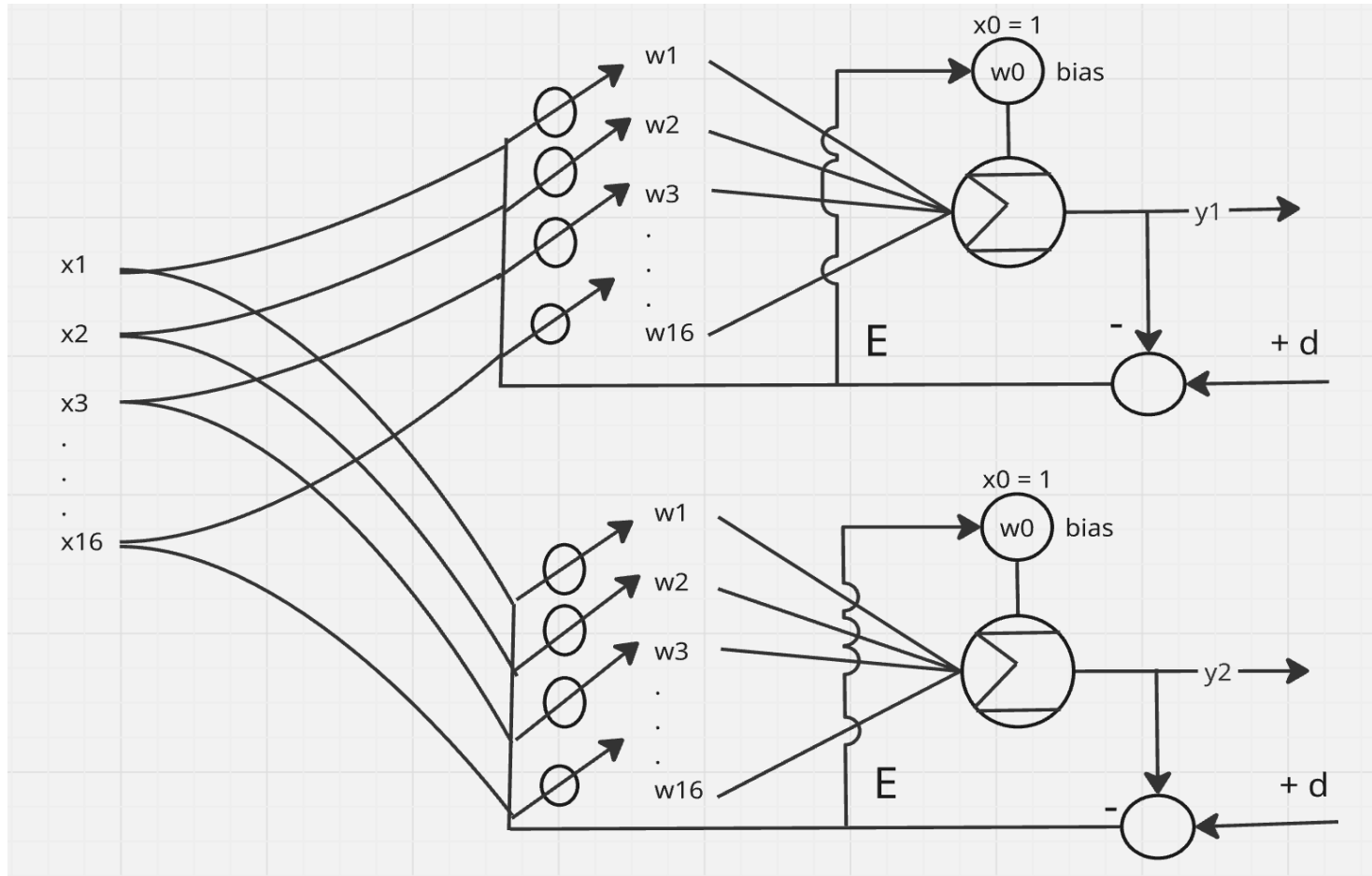
$$Y2 = W_2^t * X + b_2 = [w_{21}, w_{22}, w_{23}, \dots, w_{216}]^t * X + b_2$$

:

$$Y4 = W_4^t * X + b_2 = \dots$$

$$X = [x1, x2, x3, \dots, x16]$$

Rede Neural Artificial



Inputs e out esperados da segunda camada?

Passos (Simplificado) AdaLiNe

- **Exemplo**

- Dados: $(x_1 = 2, y_1 = 1)$ e $(x_2 = -1, y_2 = -1)$

- Pesos iniciais: $w=0.0, b=0.0$

- Taxa de aprendizado: $\eta=0.1$

- Fórmulas:

- $y = wx + b$

- $e = d - y$

Passo 1

$$y = 0$$

$$e = 1$$

$$w_{\text{next}} = 0 + 0.1 * 1 * 2 = 0.2$$

$$b_{\text{next}} = 0 + 0.1 * 1 = 0.1$$

Passo 2

$$y = 0.2 * (-1) + 0.1 = -0.1$$

$$e = (-1) - (-0.1) = -0.9$$

$$w_n = 0.2 + 0.1 * (-1) * (-0.9) = 0.29$$

$$b_n = 0.1 + 0.1 * (-0.9) = 0.01$$

Passo 3

$$y = 0.29 * 2 + 0.01 = 0.59$$

$$e = 1 - 0.59 = 0.41$$

$$w_n = 0.29 + 0.1 * 2 * 0.41 = 0.372$$

$$b_n = 0.01 + 0.1 * 0.41 = 0.051$$

Passos (Simplificado)

Passo	x	y	y_out	e	Novo w	Novo b
1	2	1	0	1	0.2	0.1
2	-1	-1	-0.1	-0.9	0.29	0.01
3	2	1	0.59	0.41	0.372	0.051
4	-1	-1	-0.321	-0.679	0.4399	-0.0169
5	2	1	0.8629	0.1371	0.4673	-0.0032
6	-1	-1	-0.4705	-0.5295	0.5203	-0.0561
7	2	1	0.9844	0.0156	0.5234	-0.0546
8	-1	-1	-0.578	-0.422	0.5656	-0.0968
9	2	1	1.0344	-0.0344	0.5587	-0.1002
10	-1	-1	-0.6589	-0.3411	0.5928	-0.1343

Condições para fim das iterações

- Coeficiente W não muda
- Erro estabiliza
- Um número de iterações pré-definido

Passos (Simplificado) Rede

- **Exemplo**
- `array([[ 0.17564817, -0.45409656, -0.07654885, 0.44196638],`
- `[ 0.6105984 , 0.35753575, 0.42663106, -0.19989887],`
- `[ 0.00656799, 0.22540088, 0.18744895, 0.44107392],`
- `[ 0.24199682, 0.25869402, 0.09813225, -0.21786803],`
- `[ 0.19807556, -0.21563004, -0.5896655 , -0.11644482],`
- `[-0.05018181, 0.3895604 , 0.23705137, 0.38910556],`
- `[-0.0422593 , 0.1488903 , 0.23784032, -0.18718903],`
- `[ 0.23637089, -0.04396867, -0.3733943 , -0.51371574],`
- `[ 0.08902457, 0.4328548 , -0.15125728, 0.44024682],`
- `[ 0.08443442, -0.35713628, 0.4220012 , -0.48862752],`
- `[-0.16093272, -0.07139882, -0.40694967, 0.12083614],`
- `[ 0.2220827 , -0.39451802, -0.14577553, 0.09657893],`
- `[-0.13722849, 0.07822154, 0.60645515, -0.38792893],`
- `[ 0.0424283 , 0.6180555 , -0.06544294, 0.37505734],`
- `[-0.45098245, -0.14265509, -0.44201308, -0.6223746 ],`
- `[ 0.16541703, 0.42564288, 0.23518936, -0.12764558]])`
- `Biases: array([-0.09814182, -0.10614556, -0.0261706 , 0.12481583])`

Rede Neural Artificial

- Nas redes neurais modernas, o método dominante para ajustar os pesos é o **Gradiente Descendente** e suas variantes (como SGD - Stochastic Gradient Descent, Adam, RMSprop, Adagrad, etc.).
- A ideia central é:
 - **Calcular o Erro:** A rede faz uma previsão e o erro é calculado usando uma função de perda (como Entropia Cruzada Categórica para classificação, ou Erro Quadrático Médio para regressão).
 - **Calcular os Gradientes:** O algoritmo de retropropagação (backpropagation) é usado para calcular o gradiente da função de perda em relação a cada peso e viés na rede. O gradiente indica a direção de maior aumento da função de perda.
 - **Atualizar os Pesos:** Os pesos e vieses são ajustados na direção oposta ao gradiente (descendo na "superfície" da função de perda) para minimizar o erro. O tamanho do passo nessa descida é controlado pela taxa de aprendizado.

CNN (Convolutional Neural Network)

- Nos modelos de deep learning com CNN para treino e teste, cada imagem de entrada passa por uma série de camadas de convolução com filtros (kernels), camadas de pooling, camadas totalmente conectadas (FC) e, ao final, aplica-se a função Softmax para classificar o objeto com valores probabilísticos entre 0 e 1.
- A convolução nada mais é do que operações matemáticas entre a matriz de entrada e um filtro (Kernel) que é uma matriz menor que a analisada, varrendo pixel a pixel, até passar por toda imagem.
- O resultado é uma matriz menor ou de mesmo tamanho, com características ressaltadas pelo filtro.
- Esse destaque/ressalte de detalhes da imagem vai fazer um melhor treinamento da rede neural.

CNN (Convolutional Neural Network)

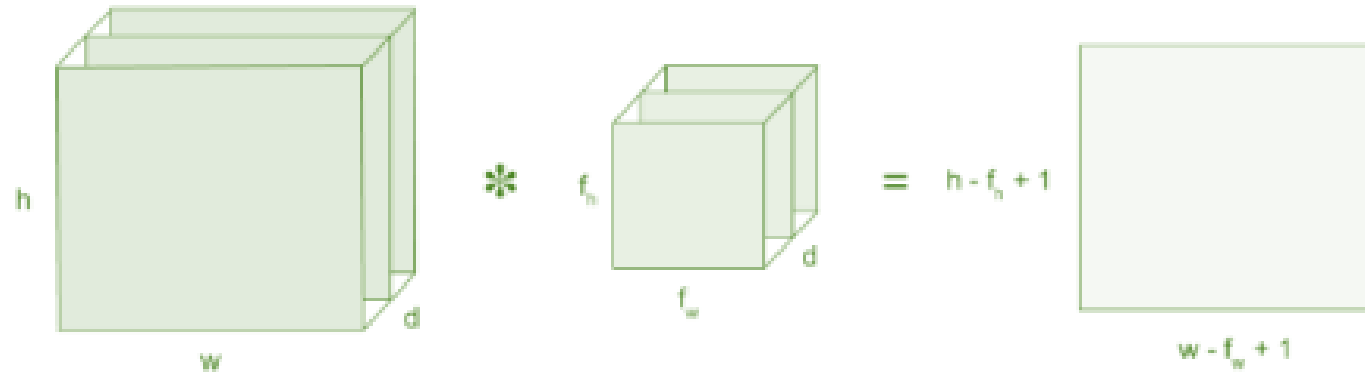
- A camada de convolução extrai padrões locais (bordas, texturas, partes de objetos) de uma imagem aplicando pequenos filtros (kernels) sobre a entrada
- Cada filtro “varre” a imagem e, em cada posição, calcula uma soma ponderada dos pixels do patch local — esse valor vira um pixel do feature map (mapa de características)
- A grande vantagem: pesos compartilhados (o mesmo kernel é usado em toda a imagem) e conectividade local (cada saída depende de um patch local)

- Entrada: dimensão $H_{in} \times W_{in} \times C_{in}$

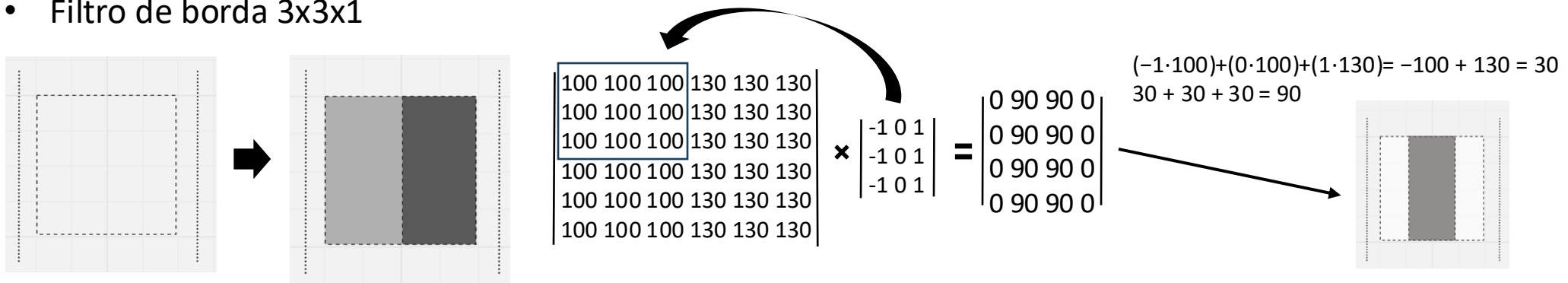
- Kernel (filtro): $K_h \times K_w \times C_{in}$

$$Y_{i,j,f} = \sum_{c=1}^{C_{in}} \sum_{u=1}^{K_h} \sum_{v=1}^{K_w} X_{i+u-1, j+v-1, c} \cdot W_{u,v,c,f} + b_f$$

CNN (Camada de Convolução)



- **Kernels 3x3** são o padrão hoje (boa relação custo/benefício).
 - Mas existem filtros 1x1, 2x2, 5x5, 7x7, 11x11 etc.
- Matriz de imagem em scala de cinza 6 x 6 x 1
- Filtro de borda 3x3x1



CNN (Convolutional Neural Network)

- A camada de convolução extrai padrões locais (bordas, texturas, partes de objetos), aplicando os filtros específicos para cada objetivo.
- O filtro “**varre**” a imagem (movendo-se pixel por pixel) e, em cada posição, faz:
 $\sum (\text{valores da imagem} \times \text{valores do filtro})$
- Multiplicação e Soma (MAC - Multiply-Accumulate)
- Stride: pula posições; stride > 1 reduz tamanho espacial
 - downsampling
- Padding: adiciona borda (zeros ou refletido) antes da convolução

Operation	Filter	Convolved Image
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

CNN (ReLU - Rectified Linear Unit)

- É uma função aplicada após passar um filtro na imagem
- Introduz não-linearidade aos modelos e aumenta a eficiência do cálculo
- Se for positivo ou zero, pode passar. Se for negativo, vira ZERO.

$$f(x) = \max(0, x)$$

12	-5	8	-20
-1	18	0	4
7	-15	25	-8
-10	3	-2	15

 $\rightarrow$

12	0	8	0
0	18	0	4
7	0	25	0
0	3	0	15

CNN (Camada de Pooling)

- Ênfase na redução de dimensionalidade, na imagem mapa de características.
- Não é uma operação matemática complexa, mas simplesmente uma seleção de máximo (MaxPool) ou cálculo de média (AvgPool).
- Implementar um MaxPool é simples: é um comparador que recebe 4 valores e seleciona o maior.

12 30 25 18 10 5

8 15 40 32 12 8

20 35 10 22 18 15

17 5 28 38 25 20

10 22 18 14 30 12

5 16 9 20 11 25

• $\text{Max}(12, 30, 8, 15) = 30$

Referências

- <https://medium.com/@RaghavPrabhu/understanding-of-convolutional-neural-network-cnn-deep-learning-99760835f148>
- <https://poloclub.github.io/cnn-explainer/>
- <https://quickdraw.withgoogle.com/>
- https://bookdown.org/jessicakubrusly/curso_de_machine_learning/book/07-redesneurais.html

Página em branco.

Aula 1

Título da Aula 1

Título do slide

- Conteúdo...

Página em branco.